

LPOO Aula 6

Introdução a Orientação a Objetos

Ana Karina D. Salina de Oliveira

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Linguagem de Programação Orientada a Objetos



Conteúdo da Aula

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- 1 Introdução a Orientação a Objetos
- 2 Classes
- 3 Objetos
- 4 Encapsulamento
- 5 Método Construtor
- 6 Variáveis

(*baseado no material do prof. Anderson Viçoso de Araujo)



Programação Orientada a Objetos (POO)

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Linguagem de Programação Orientada a Objetos

- Criada por Alan Kay em 1969
- Paradigma de programação que utiliza o conceito da criação de objetos e do relacionamento entre eles
- Provê características para o desenvolvimento rápido de software
- É o paradigma de programação muito utilizado atualmente



Introdução à POO

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Objetos

- O mundo a nossa volta está cheio de objetos
 - Carros, construções, sapatos, pessoas, celulares...
- Cada um destes objetos tem a capacidade de executar ações
 - Acelerar, correr, ligar, enviar mensagem
 - Uma ação pode afetar outros objetos

Introdução à POO

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

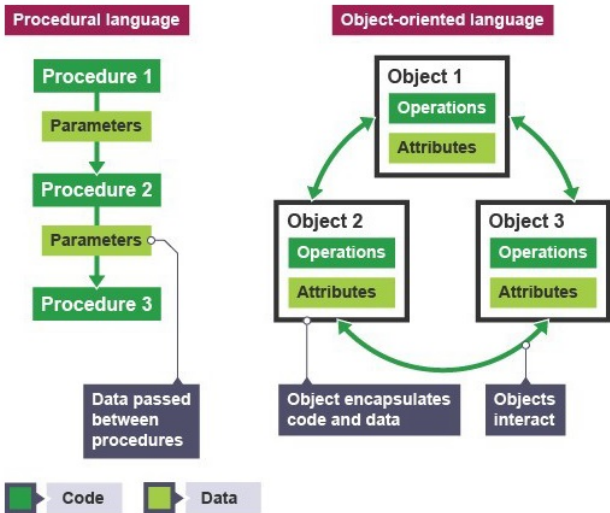
Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis





Programação Estruturada x Programação Orientada a Objetos

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Programação Estruturada

- O programa é representado como uma estrutura lógica
- O fluxo de execução do programa depende da estrutura do programa
- O código é o mais importante
- Menos segurança dos dados
- Menos reusabilidade de código
- Menos Flexível

Programação O. Objetos

- O programa é escrito como uma coleção de objetos que se comunicam entre si
- A entidade básica é o objeto. Os cálculos são realizados somente pelos objetos.
- Os dados são o mais importante
- Mais segurança dos dados
- Mais reusabilidade de código
- Mais flexível

Classes

- São modelos (template) usados para criar objetos, definindo:
 - O comportamento de seus objetos através de métodos
 - As informações destes objetos através de atributos
- Estrutura que abstrai um conjunto de objetos com características similares
 - Uma entidade lógica

Classes

- Auxiliam no Reuso – Exemplo:
 - Ao desenhar uma planta de uma casa, um arquiteto pode usar a mesma planta para construir várias casas iguais
 - Um formulário impress/copiado e distribuído para várias pessoas preencherem, o formulário é um só (molde) mas as cópias preenchidas são diferentes (objetos)
 - Reusar classes existentes, poupa tempo e esforço



Introdução à POO

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Classes

- Encapsulam propriedades (atributos) e comportamentos (métodos)
 - Logo, atributos e métodos são membros internos de uma classe
- Ao escrever código Java, você está escrevendo classes ou interfaces



Introdução à POO

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Membros Internos

- Atributos (ou Propriedades)
 - São variáveis declaradas dentro da classe, mas fora de qualquer método
- Métodos (ou comportamentos)
 - São métodos internos da classe que correspondem a comportamentos associados à classe

Responsabilidades de uma Classe

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Ao construir uma classe, temos que definir quais são as suas responsabilidades
 - O que ela faz? Quais são suas características?
 - Com quais classes ela se relaciona (colabora)?
 - Ela não pode fazer algo fora das suas responsabilidades
- Exemplo Classe ContaCorrente:
 - Apresentar o saldo
 - Possibilitar um saque
 - Possibilitar um depósito

Objetos

- São criados a partir de classes
 - São instâncias de uma classe
 - Vários objetos podem ser instanciados a partir de uma classe
- A classe é o molde e o objeto é algo que foi moldado pela classe
 - A classe é similar a planta da casa e a casa (fisicamente) é o objeto criado



Exemplo

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Classe Smartphone
 - Atributos (propriedades):
Número de processadores, memória, tamanho da tela, marca...
 - Comportamentos (métodos):
Ligar, mandar mensagem, conectar na internet, ...
- Objetos (instâncias):
 - – iPhone 6, Samsung Galaxy 5, Sony Xperia Z3, LG Nexus5, ...

Exemplo

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Métodos (Comportamentos)

```
package exemplo;
```

```
class Smartphone{
```

```
    int numeroDeProcs;  
    double memoriaRam;  
    double tamanhoTela;  
    String marca;
```

Atributos
(Propriedades)

```
    void mandarMsg(String numero, String msg){  
        //...  
    }
```

```
    void ligar(String numero){  
        //...  
    }
```

```
}
```

- Objeto Iphone 6
 - numeroDeProcs = 2
 - memoriaRam = 1
 - tamanhoTela = 4.7
 - marca = Apple

- Objeto Nexus 5
 - numeroDeProcs = 4
 - memoriaRam = 2
 - tamanhoTela = 4.95
 - marca = LG

Ciclo de Vida de um Objeto

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Um objeto é criado usando a palavra reservada `new` (instanciação)
 - `Pessoa pes = new Pessoa();`
A JVM busca o arquivo `Pessoa.class` no disco e o carrega para a memória
A JVM cria um espaço na memória do tamanho do objeto e aponta uma referência para o objeto
- Não é possível destruir objetos diretamente da memória em Java
 - Eles são removidos automaticamente da memória (pelo Garbage Collector)
Como por exemplo, quando nenhuma referência apontar para o objeto

Exemplo

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

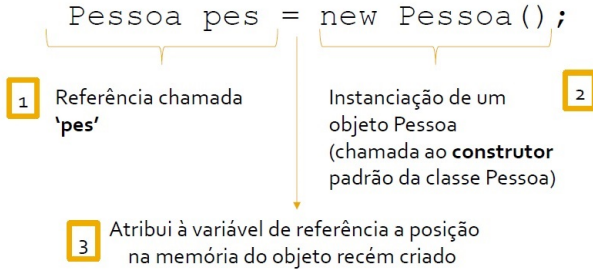
Classes

Objetos

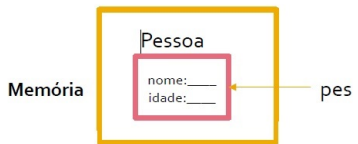
Encapsulamento

Método
Construtor

Variáveis



```
class Pessoa {
    String nome;
    int idade;
}
```



Operador . (Ponto)

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Possibilita o acesso a membros internos de uma classe
 - Mas nem sempre isso é possível, depende do modificador de acesso(publico) de cada membro
- Exemplo:

```
Pessoa p = new Pessoa();  
p.nome = "João";  
p.idade = 35;  
p.falar();
```

Encapsulamento

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Técnica para fazer com que os campos de uma classe sejam escondidos e acessá-los via métodos públicos
 - Se um campo é declarado privado, ele não pode ser acessado fora da classe, escondendo, assim, os campos dentro da classe
- Provê fácil manutenção, flexibilidade e extensibilidade para o código
- Também conhecido como Data Hiding
- A forma mais comum de uso do Encapsulamento é através dos getters e setters

Encapsulamento

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

```
public class Car{
    public float speed = 0;
    public boolean started = false;
    public boolean reverse = false;
}

public class Test{
    public static void main(String[] args){
        Car car = new Car();
        car.started = true;
        car.speed = 100;
        car.speed = 0;
        car.speed = 200;
        car.reverse = true;
        car.started = false;
    }
}
```

Encapsulamento

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Exemplo

```
class Car {  
    private float speed = 0;  
    private boolean reverse = false;  
    private boolean started = false;  
  
    public void start() {  
        if (started) {  
            // the car is already started  
        } else {  
            started = true;  
        }  
    }  
  
    public void accelerate() {  
        if (started) {  
            speed += 10;  
        } else {  
            // car is not started yet  
        }  
    }  
  
    public void break() {  
        if (started) {  
            speed -= 10;  
        } else {  
            // car is not started yet  
        }  
    }  
  
    public void off() {  
        if (started) {  
            started = false;  
        } else {  
            // car is already off  
        }  
    }  
}
```

```
class Test{  
    public static void main(String[] a){  
        Car car = new Car();  
        car.off(); //car is already off  
        car.start();  
        car.accelerate();  
        car.accelerate();  
        car.accelerate();  
        car.break();  
        car.break();  
        car.break();  
        car.off();  
    }  
}
```

Qualquer classe que queira acessar os atributos "escondidos" deve fazê-lo através dos métodos públicos

Exemplo

```
class Person{  
    private String name;  
    private int age;  
  
    public int getAge() {  
        return age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setAge(int newAge) {  
        age = newAge;  
    }  
  
    public void setName(String newName) {  
        name = newName;  
    }  
}
```

- **Getter:** Método usado para retornar o valor de um atributo privado*

```
public Tipo getNomeAtributo() {  
    return nomeAtributo;  
}
```

- **Setter:** Método usado para alterar o valor de um atributo privado*

```
public void setNomeAtributo(Tipo novo) {  
    nomeAtributo = novo;  
}
```

*Em geral privado

```
public static void main(String args[]){  
    Person p = new Person();  
    p.setName("James");  
    p.setAge(20);  
  
    System.out.print("Name : " +  
p.getName() + " Age : " + p.getAge());  
}
```

Resumo

- Os campos de uma classe podem ser somente leitura ou somente escrita
 - Ou seja, podemos ter somente um dos métodos getter ou setter
- Uma classe tem total controle sobre o que está armazenado nela
- Quem usa a classe não sabe como a classe armazena seus dados

- Toda vez que a palavra reservada `new` for usada, um método construtor vai ser chamado
 - Ele serve para “construir” um objeto
 - Pode ter parâmetros que indicam como esse objeto deve ser inicializado
 - Não inicializa os atributos automaticamente
- Se uma classe não possuir um construtor, o Java cria um construtor padrão (sem parâmetros) para a classe automaticamente

Método Construtor - Exemplo

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

```
class Pessoa {  
    String nome;  
    int idade;  
    Pessoa() {  
    }  
    Pessoa(String novoNome, int novaIdade) {  
        nome = novoNome;  
        idade = novaIdade;  
    }  
}
```


Declaração de Variáveis

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Variáveis servem para armazenar valores
- Existem dois tipos de variáveis:
 - Tipos Primitivos:
char, boolean, byte, short, int, long, double ou float
 - Variáveis de referência:
Usadas para se referir e acessar um objeto



Variáveis

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Tipos primitivos ou variáveis de referência podem ser:

- **Atributos:**
 - São variáveis declaradas dentro da classe, fora de qualquer método. Possuem valores padrão de inicialização
- **Variáveis locais:**
 - Variáveis declaradas dentro de um método
 - É obrigatório inicializá-las antes de usar
- **Parâmetros:**
 - São variáveis declaradas dentro da assinatura de um método

Variáveis Locais

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- São aquelas declaradas e inicializadas dentro de um método/escopo
 - E destruídas dentro do método/escopo
 - Com isso, não podem ser acessadas em um código fora de onde foram declaradas
- Embora, o valor da variável local possa ser passado para outro método, armazenando seu valor em uma variável de instância, a variável propriamente dita só vive dentro do escopo do método

- Devem sempre ser inicializadas antes de serem usadas

```
class TimeTravel{  
    public static void main(String[] args) {  
        int year = 2050;  
        System.out.println("The year is"+year);  
    }  
}
```

Variáveis de Instância (Atributos)

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- São definidas dentro da classe, mas fora de qualquer método
 - Não têm posição específica, mas em geral ficam no começo da classe
- São inicializadas quando a classe é instanciada
- São campos que pertencem a cada objeto único
- Exemplo a seguir define as variáveis de instância nome, titulo e gerente para o objeto Funcionario

```
class Funcionario {  
    String nome;  
    String titulo;  
    String gerente;  
}
```

Variáveis de Instância de Referência a Objetos

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- Quando comparadas com as variáveis primitivas não inicializadas, as variáveis de referência são diferentes

```
class Livro{
    String titulo;//variável de referência de instância
    String obterTitulo () {
        return titulo;
    }
    public static void main(String[] args) {
        Livro liv = new Livro();
        System.out.println("Título:" + liv.obterTitulo());
    }
}
```

Valores Padrão para Variáveis de Instância

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

Tipo	Valor
boolean	false
byte	0
short	0
int	0
long	0l
char	\u0000
float	0f
double	0d
Object	null

Variáveis de Classe

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- São definidas dentro da classe, mas fora de qualquer método
- Devem ser precedidas pelo modificador static
- Pertencem a classe onde foram declaradas, consequentemente a todos os objetos instanciados a partir daquela classe
 - Podem ser utilizadas sem ter que instanciar a classe a que pertencem

Variáveis de Classe - Exemplo

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

```
class StaticDemo {
    static int count = 0;
    void increment() {
        count++;
    }
    public static void main(String args[]) {
        StaticDemo obj1 = new StaticDemo();
        StaticDemo obj2 = new StaticDemo();
        obj1.increment();
        obj2.increment();
        System.out.println("Obj1:count " + obj1.count);
        System.out.println("Obj2:count " + obj2.count);
        System.out.println("Obj:count " + StaticDemo.count);
    }
}
```

Métodos de Classe

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

- São métodos que estão associados diretamente com a classe e não com cada instância
 - Podem ser utilizados sem ter que instanciar a classe a que pertencem
- Devem ser precedidos pelo modificador static

Métodos de Classe - Exemplo

LPOO Aula 6

Ana Karina

Introdução a
Orientação a
Objetos

Classes

Objetos

Encapsulamento

Método
Construtor

Variáveis

```
class StaticDemo2 {
    int x = 10;
    static int count = 0;
    void increment() {
        count++;
    }
    static void decrement() {
        count--;
    }
    public static void main(String args[]) {
        StaticDemo2 obj1 = new StaticDemo2();
        obj1.increment();
        StaticDemo2.increment(); //Erro:
tentando acessar um método da instância
        obj1.decrement();
        StaticDemo2.decrement();
    }
}
```

- São definidas através do uso de duas palavras reservadas da linguagem Java: `static` e `final`
 - `static`: indica que a variável é da classe, ou seja, só existe um valor para todos os objetos criados a partir daquela classe
 - `final`: indica que o valor de uma variável não pode ser alterada
- O valor das variáveis `final` não pode ser modificados, nem através de operadores `acréscimo` e `decrécimo` (`++`, `--`)
- Exemplo:
`static final double PI = 3.141592653589793;`

- Classe Aluno
 - especificar atributos para alunos
 - criar o construtor
 - especificar métodos como: fazerMatricula, calcularMedia, etc
 - inserir métodos get e set para Aluno
 - fazer o método imprimirAluno
- Classe Teste
 - inserir o método main
 - adicionar objetos da classe aluno
 - alterar atributos de aluno com método set
 - utilizar os métodos da classe aluno